

Why I Chose Isabelle/HOL

Autoformalizing Higher-Order Metaphysics

7 September 2026; core status updated 9 September 2026

I chose Isabelle/HOL because the task was to formalize the metatheory of particular higher-order logics: their languages, proof systems, and models, together with the relations between them. Classical higher-order logic is a natural metalanguage for that task. Isabelle’s inductive definitions and locales support explicit representations with visible assumptions; Sledgehammer and Nitpick support proof search and counterexample investigation; and Isar provides a readable structure for the resulting proofs. Those are methodological reasons for the original choice.

Lean 4 can support this work too. Its dependent types, metaprogramming, and visible ecosystem for language-model-assisted formalization are genuine attractions. My choice gives greater weight to Isabelle’s fit for exploratory logical metatheory and its established combination of symbolic tools. It does not depend on the amount of code subsequently written in Isabelle, and I have not run a controlled benchmark establishing that Isabelle agents outperform Lean agents on this task.

The comparison requires three distinctions: the host logic versus the represented object theory; deep versus shallow embedding; and proof correctness versus correctness of the formalization. The main case is the standalone Bacon-Dorr core repository. Two external comparisons, Goodman’s Purity-of-Pure question and Kirchner’s implementation of Zalta’s Abstract Object Theory (AOT), illustrate the same distinctions; neither application is included in this standalone repository. The external-tool assessment retains its date of 7 September 2026, while the core status below has been updated through 9 September.

1 Foundations and the choice of representation

Isabelle/HOL implements classical higher-order logic with simple types, within Isabelle’s generic logical framework. This makes it a congenial metalanguage for classical reasoning about typed higher-order languages, models, and proof systems. Its datatypes, inductive definitions, recursive functions, and locales provide established means of expressing the relevant structures and their assumptions [1]. Lean 4 is based on dependent type theory. Classical reasoning is available in Lean; using it does not require recasting the source theory as constructive mathematics [2]. Both systems can support the metatheory at issue.

The closer fit of Isabelle/HOL is nevertheless a fit of *metalanguage* to task. It does not identify Bacon’s logic with Isabelle’s HOL. In particular, we must not simply identify source propositions with HOL Booleans, source identity with equivalence of truth values, or every higher-type domain with a full function space. Those choices can settle substantive issues of extensionality or granularity before the source axioms have been represented. The same discipline is necessary in Lean. Neither host establishes the adequacy of an embedding merely by checking its proofs.

In a *deep embedding*, expressions are data, and typing and derivability are independently defined relations on those data. The source-facing core uses explicit named terms, typing, declared signatures, capture-avoiding substitution, conversion, and independently defined deduction. Earlier de Bruijn and parametric representations remain as supporting layers, with their source correspondences proved separately. Semantic interpretation is a further construction. Thus a statement of the schematic form

$$S \vdash_H A \quad \leftrightarrow \quad S \models_{\mathcal{M}} A$$

relates two separately specified notions. Here S is a theory of sentences, A is a sentence, and $\mathcal{M}$ is a specified model class. The typing, language, and model-class qualifications are part of the theorem.

This architecture is useful when the research concerns substitution, admissible rules, derivability, soundness, or completeness. It also incurs substantial work: an agent must learn the local syntax and prove the structural lemmas that connect its layers. Isabelle supports that work well; it does not make it automatic. Lean could support the same extrinsic representation. The choice is not between an Isabelle system capable of metatheory and a Lean system incapable of it.

2 Proof search, countermodels, and auditability

Sledgehammer supplies an unusually mature connection between interactive proof and external automated provers. It selects relevant facts, translates proof obligations, and suggests proofs for replay in Isabelle. It is particularly useful after the conceptual work has reduced a goal to a consequence of structural lemmas. An external prover’s success remains separate from successful proof reconstruction; the accepted artifact should be the replayed Isabelle proof [3].

Nitpick contributes a complementary form of investigation: searching for counterexamples to proposed HOL statements. In a metaphysics development, this can expose an

omitted assumption in a modal inference, an overstrong identity principle, or a proposed semantic condition that fails in a small structure. Its finite searches and approximations must be interpreted with their reported status. A potential counterexample requires checking, and failure to find one establishes neither validity nor consistency. Moreover, a deep embedding can make the relevant search difficult: a finite test of a semantic fragment does not automatically instantiate all the closure and schema conditions of the source theory [4].

Lean has substantial symbolic automation too. Its core `grind` tactic, for example, combines several forms of automated reasoning; Lean Copilot can combine generated tactics with Aesop [5, 6]. Isabelle’s advantage here is the established integration of proof search and model finding, especially for exploratory logical metatheory. We should not convert that practical advantage into a universal ranking of provers.

Isar also supports an auditable presentation: named assumptions, intermediate claims, and explicit proof structure can accompany a source locator and a notation dictionary [7]. That is valuable when a reader must check why a formal lemma represents a paragraph in a paper. Lean supports readable structured proofs as well. In either system, source correspondence must be written and maintained; it is not a consequence of choosing an attractive surface notation.

3 Lean’s advantages for autoformalization

The strongest current argument for Lean is the surrounding AI infrastructure. LeanDojo established tools for extracting premises, tactics, and proof states and for interacting with the prover. Its current repository directs new work to LeanDojo-v2, whose proof-search interface uses Pantograph to apply tactics and inspect resulting goals [8]. Lean Copilot supplies tactic suggestion, proof search, and premise retrieval inside Lean [6]. These are concrete examples of the generate-check-repair cycle that an agent needs.

Statement autoformalization has a substantial Lean-based research literature. ProofNet originally paired informal mathematics with *Lean 3* statements [9]; it should not be described as originally a Lean 4 benchmark. Subsequent work evaluates Lean 4 translations, including corrected ProofNet data and the research-level RLM25 benchmark [10]. Taken together, these tools and benchmarks support the judgment that Lean currently offers a broader visible LLM/autoformalization ecosystem. They do not tell us the composition of any proprietary model’s training corpus, nor do they establish a numerical advantage for higher-order metaphysics.

Lean’s language design is a further attraction. Macros and elaborators permit custom notation to be translated into checked terms, and tactics can be implemented in Lean it-

self [11]. This makes it convenient to build a domain language, expose proof states, and connect search to elaboration. Isabelle has its own extensive customization facilities, including Isabelle/ML and Eisbach [1, 7]. The advantage is one of developer workflow and available integrations, not an exclusive capability of Lean.

Dependent types also permit *intrinsically typed syntax*. Schematically, one can define a family $\text{Term}(\Gamma, \tau)$, whose members are terms of object type τ in context Γ . Application and abstraction then have shapes such as

$$\begin{aligned}\text{app} &: \text{Term}(\Gamma, \sigma \rightarrow \tau) \times \text{Term}(\Gamma, \sigma) \rightarrow \text{Term}(\Gamma, \tau), \\ \text{lam} &: \text{Term}(\sigma :: \Gamma, \tau) \rightarrow \text{Term}(\Gamma, \sigma \rightarrow \tau).\end{aligned}$$

These are representation sketches, not executable Lean declarations. Context-indexed variables would supply the variable case. Ill-typed applications cannot be constructed in such a representation.

This can remove separate preservation obligations, but dependent substitution and equality transport may become more involved. More importantly, a source with an explicit typing relation may call for a theorem connecting that relation to the intrinsic representation. Intrinsic typing need not hide the source machinery: we can retain raw syntax and prove an adequacy bridge. For this task, I preferred an explicit extrinsic representation because the source typing judgments and their preservation properties are themselves objects of investigation. That preference concerns what the formalization should expose, rather than the cost of changing an existing implementation.

4 Why project-specific embeddings change the comparison

A model familiar with ordinary Lean mathematics can reuse conventions and library lemmas with little adaptation. It cannot thereby know the intended meaning of this project's `Forall`, `Lam`, or `H_proves`, nor which conversion or consequence relation a source theorem requires. A deep embedding in Lean would introduce similarly project-specific definitions. General skills in induction, tactics, and library use would still transfer, but much of the critical knowledge would have to come from the project.

For this reason, retrieval and source fidelity are likely to dominate many of the difficult tasks here. Useful context includes the exact statement, its source passage, the permitted imports, typing and signature conditions, and nearby proofs at the same logical strength. The standalone repository has a notation guide, an explanatory source table, and an Isabelle-native dependency graph for the checked core. These are practical foundations for an agent

that retrieves the right mathematical context. The larger parent research project keeps its Bacon-Dorr-Goodman and AOT graphs separate.

This is a reason to expect a reduction in Lean’s generic pretraining advantage, not evidence that the advantage disappears. Research on Lean statement autoformalization itself finds difficulty with research-level material without appropriate context [10]. A direct comparison on previously unseen tasks from this repository would still be needed. Existing retrieval tools would also need adaptation: Lean Copilot’s documented premise selector uses a fixed Lean/mathlib snapshot, rather than automatically indexing any new embedded theory [6].

5 The core development and two external comparisons

5.1 Bacon and Bacon-Dorr: explicit logical metatheory

Bacon’s higher-order logics and the Bacon-Dorr treatment of Classicism make the source language, inferential rules, and semantic assumptions central to the formalization [12, 13]. Isabelle is a good fit for keeping these objects separate while proving their connections. Binding, substitution, signatures, deduction, and model construction were therefore central requirements when choosing a prover, rather than just infrastructure to preserve after making the choice.

The standalone repository’s September 9 checkpoint contains soundness, sentence-set model existence, and closed strong completeness for the Bacon-Dorr named H presentation. The book’s full minimal H language also has general-model soundness, original-signature model existence, and global strong completeness, including open formulas and arbitrary premise sets. The statements retain their explicit variable-stock, language, model-class, and carrier qualifications.

For the paper’s relational-type Classicism, category soundness/completeness and single-formula action-model soundness/completeness are checked at the stated carrier scopes. The book’s full-type C development has an actual canonical modal model and original-signature model existence for countably declared signatures. Generic interpretation existence is now checked for every independent full-minimal modal model: typed K/S abstraction elimination supplies values in the chosen domains and proves the literal future-function abstraction clause. No supplied interpreter or extra nonemptiness assumption is added.

Generic full-type C soundness and final unrestricted modal completeness remain open. The HOL-ZF model representation retains its explicit foundation and documented future-restricted implication convention. These boundaries are recorded in the standalone source

guide and verification records. This comparison note summarizes those checked endpoints; it supplies no new proof certificate.

5.2 Goodman: preserving the exact question

Goodman’s question asks whether Purity of Pure is consistent with the remaining theory and a unique fundamental proposition, without assuming Purity of Fun [14]. Purity of Pure says that the purity property itself is pure; the logical stock and closure principles determine what this requires at the relevant types. A contradiction obtained by adding Purity of Fun would answer a stronger question. Likewise, a small structure satisfying selected instances would not establish a model of the unrestricted theory.

The parent research project’s record leaves the central question open. It separates the canonical reconstruction of Bacon’s appendix model, Goodman-specific interpretations on those carriers, and secondary experimental models. Some results use HOL-ZF with axiomatized ZFC and retain that relative semantic basis. These distinctions illustrate why both proof checking and countermodel search need source-sensitive interpretation. Keeping Goodman in Isabelle allows it to reuse the exact background development, with explicit bridges wherever alternative presentations are involved.

5.3 Kirchner and Zalta: a shallow embedding with a restricted interface

Kirchner’s AOT environment uses a *shallow semantic embedding*: object expressions are represented through semantic objects in HOL, and much of the binding infrastructure uses HOL functions. There is no general formula datatype and independently reified derivation relation analogous to the Bacon development. Zalta’s *Principia Logico-Metaphysica* supplies the target presentation [15]. This substantially reduces the amount of explicit syntax metatheory required for proving theorems within AOT.

However, the embedding is not a simple identification of AOT with HOL. AOT includes hyperintensional relations, free logic, and the distinction between exemplification and encoding. Kirchner supplies custom notation and proof commands, together with an abstraction layer that restricts automation to AOT axioms and inferential rules. This is intended to exclude theorems that merely reflect details of the semantic implementation [16].

A Lean implementation could also exploit host binding, types, and custom elaboration. Thus shallow embedding narrows the architectural difference between the provers, while leaving substantial semantic and interface work. The AOT case illustrates what a different choice of embedding can accomplish in Isabelle; it is not the basis for the choice of a deep embedding here. That external AOT session extends HOL-Cardinals independently and

is not part of this standalone core repository. A shared host prover does not supply a mathematical bridge between the theories.

6 A concise comparison

Table 1 summarizes the practical tradeoffs. The entries describe available support and this repository’s needs, rather than measured success rates.

Issue	Isabelle/HOL	Lean 4
Host foundation	Classical simple HOL; close metatheoretic fit	Dependent type theory; classical reasoning available
Deep embedding	Mature syntax, deduction, and model infrastructure	Equally viable; intrinsic typing is an additional option
Automation	Sledgehammer and integrated Nitpick exploration	Strong tactics; extensive AI integrations
LLM workflows	Smaller ecosystem; recent proof-state and search work	Broader visible tooling and benchmark ecosystem
Customization	Isabelle/ML, Eisbach, Isar, custom syntax	Macros, elaborators, and tactics written in Lean
Source audit	Structured Isar proofs with named assumptions	Structured proofs and source maps can be built
Shallow embedding	External AOT implementation illustrates the approach	Host binding and elaboration also support the approach
Fit for this task	Classical metatheory with integrated proof and counterexample tools	Viable metatheory with dependent types and extensive AI tooling

Table 1: Tradeoffs for autoformalizing higher-order metaphysics.

7 What the choice of tool does not settle

The special difficulty in autoformalizing philosophy is that the intended formal statement may itself be at issue. A proof assistant can accept a proof of a perfectly well-formed statement that misrepresents the source. This occurs in mathematics too, as recent work on evaluating statement autoformalization emphasizes [10]; philosophy adds sustained questions about the interpretation of the primitives, axioms, and semantics. In this setting, a higher proof-success rate is useful only when it measures success on the right formalization.

For example, identifying necessarily equivalent propositions when the target theory distinguishes their identity can erase the intended hyperintensional structure. Restricting a

quantifier to an unintended stock can make a schema easier to prove. Importing a stronger calculus can turn an open derivability question into an immediate consequence. A kernel checks the resulting formal argument; it does not compare these choices with the philosophical text. An effective workflow must therefore audit the translation as well as the proof.

The resulting workflow should keep four obligations explicit:

1. **Fix the target.** Record the source passage, formal statement, logical strength, signature, and semantic scope before generating a proof. Have the source correspondence reviewed separately from tactic success.
2. **Retrieve within that scope.** Use the notation dictionary, source matrix, theorem statements, and the appropriate Isabelle-native graph. Keep the independent theory families separate and expose the assumptions of candidate premises.
3. **Check and repair incrementally.** Give the agent structured proof states and errors; combine short proof attempts with Sledgehammer and, where suitable, Nitpick. Retain replayable proofs and validate candidate models. Serialize Isabelle builds in accordance with the repository's existing rules.
4. **Evaluate on faithful results.** Use held-out source passages and lemmas from the repository. Record translation accuracy, completed checked proofs, premise discipline, human corrections, and time or cost. Do not count an easier substituted statement as a successful formalization.

There is evidence that such a pilot is feasible. The IsaMini line of work studies a simplified proof language combined with Isabelle automation; Stepwise exposes fine-grained Isabelle proof states through a REPL and combines learned search with symbolic repair [17, 18]. These are research demonstrations on their own tasks. They do not establish compatibility with the local toolchain or success on metaphysical source translation, which the pilot would need to test.

My reason for choosing Isabelle was the combination of a suitable metalanguage, mature proof and counterexample tools, and a proof format that makes assumptions and intermediate reasoning inspectable. These are useful features before any project-specific development exists. Lean's advantages remain relevant, especially when a project's main objective is to use the broadest available AI formalization infrastructure. Neither choice replaces the need to check that the formalization expresses the intended theory. That requirement is central to the project, whichever prover one uses.

Repository basis. The core status was refreshed from STATUS.md, docs/SOURCE_CORRESPONDENCE.md, and docs/VERIFICATION.md at the standalone repository's checked 9 September 2026 checkpoint 41bb91f. The main H and C scopes and generic interpretation-existence audit are recorded there. The original comparison's external-tool assessment and bibliography date from 7 September; no new comparative benchmark or external-agent compatibility test was run for this revision. The parent report is retained unchanged.

<https://github.com/fitelson/bacon-dorr-isabelle>

References

- [1] Isabelle. *Documentation for Isabelle2025-2*.
<https://isabelle.in.tum.de/documentation.html>.
- [2] Lean. *The Lean Language Reference*, “The Type System” and “Axioms.”
<https://lean-lang.org/doc/reference/latest/>.
- [3] Jasmin Blanchette, with contributions from Martin Desharnais, Lawrence C. Paulson, and Lukas Bartl (2026). *Hammering Away: A User’s Guide to Sledgehammer for Isabelle/HOL*.
<https://isabelle.in.tum.de/doc/sledgehammer.pdf>.
- [4] Jasmin Blanchette (2026). *Picking Nits: A User’s Guide to Nitpick for Isabelle/HOL*.
<https://isabelle.in.tum.de/doc/nitpick.pdf>.
- [5] Lean. *The Lean Language Reference*, “The grind Tactic.”
<https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [6] LeanDojo contributors. *Lean Copilot: project and usage documentation*.
<https://github.com/lean-dojo/LeanCopilot>.
- [7] Makarius Wenzel. *The Isabelle/Isar Reference Manual*.
<https://isabelle.in.tum.de/doc/isar-ref.pdf>.
- [8] LeanDojo contributors. *LeanDojo and LeanDojo-v2: project documentation*.
<https://github.com/lean-dojo/LeanDojo>;
<https://github.com/lean-dojo/LeanDojo-v2>.
- [9] Zhangir Azerbayev et al. (2023). “ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics.” <https://arxiv.org/abs/2302.12433>.
- [10] Auguste Poiroux, Gail Weiss, Viktor Kunčák, and Antoine Bosselut (2025). “Reliable Evaluation and Benchmarks for Statement Autoformalization.” *EMNLP 2025*, pp. 17947–17969.
<https://aclanthology.org/2025.emnlp-main.907/>.
- [11] Lean. *The Lean Language Reference*, “Elaboration and Compilation,” “Tactic Proofs,” and “Notations and Macros.” <https://lean-lang.org/doc/reference/latest/>.
- [12] Andrew Bacon (2024). *A Philosophical Introduction to Higher-Order Logics*. Routledge.
<https://doi.org/10.4324/9781003039181>.
- [13] Andrew Bacon and Cian Dorr (2022). “Classicism.” Draft, 1 July 2022.
<https://andrew-bacon.github.io/papers/Classicism.pdf>. Numbering follows the draft specified in the repository’s source guide.
- [14] Jeremy Goodman (2026). “Is Purity of Pure consistent with Bacon’s Logical Combinatorialism?” Project notes, July 2026. Unpublished source retained in the parent research project, not redistributed here.

- [15] Edward N. Zalta (2026). *Principia Logico-Metaphysica*. Draft/excerpt, 15 May 2026, with critical theoretical contributions by Daniel Kirchner and Uri Nodelman. Source copy retained in the parent research project, not redistributed here.
- [16] Daniel Kirchner (2024). “Computer-Verified Foundations of Metaphysics.” *KI – Künstliche Intelligenz* 38:95–98. <https://doi.org/10.1007/s13218-024-00834-z>. The maintained implementation is at <https://github.com/ekpyron/AOT>; it is independent of the standalone Bacon-Dorr core.
- [17] Qiyuan Xu, Renxi Wang, Peixin Wang, Haonan Li, and Conrad Watt. “A Minimalist Proof Language for Neural Theorem Proving over Isabelle/HOL.” arXiv:2507.18885, initially submitted 2025; current revision. <https://arxiv.org/abs/2507.18885>.
- [18] Baoding He et al. (2026). “Stepwise: Neuro-Symbolic Proof Search for Automated Systems Verification.” arXiv:2603.19715. <https://arxiv.org/abs/2603.19715>.